# Mohar — Beginner Study Guide

Everything you need to explain this project in a viva, hackathon, demo, or interview. Every fact below was read from the actual code in this repository.

## 1 · What is this project?

### In the simplest words

Mohar is a computer system that keeps a **permanent, unchangeable record** of who handled an examination question paper, when, and where. Every time a sealed bundle of papers changes hands, the person doing it records that act on a phone or computer, and the system saves it in a way that **nobody can secretly edit later** — not even the people who run the system.

### What problem does it solve?

> Normally **proving who leaked an exam paper** is difficult because **the record of who held the papers is kept in handwritten registers, phone photos, and people's memory — which does not stand up in court**.
> This project helps by **making every handover digitally signed and permanently chained, so the record can be checked by anyone and cannot be quietly changed**.

From `docs/00-overview.md`: between 2015 and 2026 roughly **148** exam-fraud incidents were documented across **21** Indian states, producing **one** conviction. About **70%** were paper leaks. The document's conclusion is the key idea of this whole project: the failure is **not detection** — leaks are usually found — it is **evidence and attribution**.

### Who uses it?

| User | What they do |
|---|---|
| District officer | Seals the paper bundle, records the seal number, hands it to a courier |
| Courier | Carries the package; records pickup and delivery |
| Custodian | Keeps the package overnight in a strongroom |
| Centre superintendent | Receives, opens, prints, and confirms key destruction |
| Control-room operator | Watches everything on a dashboard and investigates problems |

> **Important honesty point.** Only the **control room** (the operator dashboard) is actually built as a working screen. The apps for couriers and centres exist as empty folders — `apps/field-app` and `apps/centre-client` contain no working UI. In the demo, the field actions come from a seeding tool that talks to the real engine.

### Why is it useful?

- An investigator can say exactly **who held the package between 21:40 and 06:15**.
- If someone edits an old record, the system **shows that it was edited**.
- If a record is **missing**, that gap is reported — silence is treated as a problem, not as "everything fine".

### What does the user actually do?

A control-room operator opens the website, looks at the dashboard, sees which packages are moving and which access attempts were refused, clicks into one package to read its full history event by event, and checks that the record has not been tampered with.

### 2-line explanation (memorise this)

> "Mohar is a tamper-evident custody chain for exam papers. Every handover is digitally signed and permanently linked, so the record of who held the paper can be proved and cannot be secretly changed."

### 1-minute explanation (speak this)

> "Exam paper leaks in India are usually detected, but almost never convicted. The reason is that the chain of custody — who held the sealed papers and when — is kept on paper registers and phone photos, which cannot be proved in court. Mohar fixes that. Every time a package changes hands, the act is signed by the device of the person doing it and written into an append-only ledger, where each record is mathematically linked to the one before it. If anyone edits an old record, every later link breaks and the tampering becomes visible. On top of that, opening a package needs a special key that only works for one stage and only for six hours, and the system runs fifteen separate checks before allowing it. The control room shows all of this live. We do not claim it stops leaks — we claim it makes leaking traceable."

## 2 · Complete feature list

These are the features that actually exist in the code. Small UI details are not listed as features.

| # | Feature | Where it lives in the code | Status |
|---|---|---|---|
| 1 | Append-only hash-chained ledger | `services/ledger`, `infra/migrations` | Working |
| 2 | Digital signatures on every event | `packages/crypto-core` | Working |
| 3 | Rotating custody keys (6-hour) | `crypto-core/custody-key.ts`, `domain/keys.ts` | Working |
| 4 | Access decision engine (15 checks) | `domain/policy.ts` | Working |
| 5 | Custody projection + 7 anomaly detectors | `domain/custody.ts` | Working |
| 6 | Activity ledger (evidence view) | `domain/activity.ts` | Working |
| 7 | Chain integrity verification | `/verify/chain` | Working |
| 8 | Merkle anchors + inclusion proof | `/anchors`, `/verify/inclusion/:id` | Working |
| 9 | Control room dashboard (7 screens) | `apps/control-room` | Working |
| 10 | Animated landing page | `control-room/src/landing` | Working |
| 11 | Device enrolment & revocation | `/devices` | Working |
| 12 | Shamir 3-of-4 key splitting | `crypto-core/shamir.ts` | Code exists, **not connected** |

### Feature 1 — Append-only hash-chained ledger

**What is it?** A list of records where nothing can ever be edited or deleted.

**What does it do?** It stores every custody act (seal, handover, receive, open, print, destroy) forever, and links each record to the one before it.

**Why do we need it?** Because if records can be edited, a guilty person can simply change the history and the evidence disappears.

**How is it used?**

```
Person performs a handover → device signs it → sent to ledger → saved forever → operator can read it
```

**Real-life example:** A school attendance register written in permanent ink where every page is numbered and glued to the previous one. You can add a page, but you cannot remove or rewrite one without it showing.

**What happens technically?** Each record gets a fingerprint called a hash. The new record's hash is calculated from the previous record's hash plus its own content: `hash = SHA256(prevHash ‖ bodyHash)`. Change any old record and every hash after it stops matching.

**Technical term:** Hash chain / append-only ledger.

> **Viva:** "It is an append-only ledger where every record is cryptographically linked to the previous one, so editing history breaks the chain and becomes visible."

**Remember:** (1) Nothing is ever edited. (2) Each record contains the previous record's fingerprint. (3) Tampering is detectable, not preventable.

### Feature 2 — Digital signatures on every event

**What is it?** An electronic signature proving which device created a record.

**What does it do?** It attaches proof of authorship to every custody act.

**Why do we need it?** A hash proves the record did not change, but not *who wrote it*. Without signatures, anyone could add fake records.

**How is it used?**

```
Device has a secret private key → signs the record → server checks it with the public key
```

**Real-life example:** Your handwritten signature on a form. Others can recognise it, but only you can produce it.

**What happens technically?** The project uses **Ed25519** signatures over **RFC 8785 canonical JSON**. Canonical JSON means the data is always written in one fixed order, so the same information always produces the same fingerprint even after it goes into the database and comes back.

**Technical term:** Public-key digital signature; JSON canonicalisation.

> **Viva:** "Every event is signed with Ed25519 over canonical JSON, so we know which enrolled device created it and the signature still verifies after a database round trip."

**Remember:** (1) Private key signs, public key verifies. (2) Canonical JSON keeps the bytes stable. (3) Signature = who; hash = unchanged.

### Feature 3 — Rotating custody keys (six-hour epochs)

**What is it?** A short code that an officer must present to act at a stage, which stops working after six hours.

**What does it do?** It limits who can do what, and for how long.

**Why do we need it?** If a key is stolen or photographed, it becomes useless very quickly.

**How is it used?**

```
Control room issues key → officer types it at the stage → engine checks it → allowed or refused
```

**Real-life example:** A hotel room card that stops working at checkout time.

**What happens technically?** Time is divided into 6-hour blocks called epochs: `epoch = floor(unixSeconds / 21600)`. A key stores which epoch it belongs to. When presented, the system compares that number with the current epoch. There is **no background job** that expires keys — they expire because time moves. The key itself is never stored, only its SHA-256 hash, exactly like a password.

**Technical term:** Time-bounded credential; expiry by arithmetic.

> **Viva:** "Keys are scoped to one package and one stage and are valid for one six-hour epoch. Expiry is arithmetic against the clock, so there is no scheduler whose failure could leave keys valid."

**Remember:** (1) 6 hours = 21600 seconds. (2) Only the hash is stored. (3) Broken infrastructure denies access instead of granting it.

### Feature 4 — Access decision engine (15 checks)

**What is it?** The function that decides whether a sealed package may be opened.

**What does it do?** It runs fifteen separate tests and only allows access if **every one** passes.

**Why do we need it?** Opening a paper bundle is the highest-risk moment. One check is not enough.

**How is it used?**

```
Officer requests access → 15 checks run → attempt is SAVED FIRST → then allowed or refused
```

**Real-life example:** Airport security — boarding pass, ID, bag scan, metal detector. Failing any one stops you.

**What happens technically?** The 15 checks are: key presented, key valid, key in window, key stage match, device enrolled, device binding, roster membership, role permitted, geofence, geo accuracy, custody window, clock skew, seal serial, package state, exam active. The engine **never stops at the first failure** — all fifteen always run, so the full shape of a suspicious attempt is recorded.

**Technical term:** Deny-by-default policy engine.

> **Viva:** "It is deny-by-default. Fifteen checks always run without short-circuiting, and the attempt is written to the ledger before the caller is told the outcome, so a denied attempt cannot be hidden."

**Remember:** (1) Deny first, allow only if all pass. (2) Every check always runs. (3) The attempt is recorded before the answer is given.

### Feature 5 — Custody projection and anomaly detection

**What is it?** The system calculates where a package is by replaying its history, instead of storing a "status" column.

**What does it do?** It works out the current state and finds problems automatically.

**Why do we need it?** A stored status can be wrong or falsified. A calculated status always matches the evidence.

**How is it used?**

```
Operator opens a package → system replays all its events → shows state + list of anomalies
```

**Real-life example:** Instead of trusting a bank's printed balance, you re-add every transaction yourself.

**What happens technically?** `projectCustody()` reads events in order and derives state, current holder, and anomalies. The seven detectors are: `illegal_transition`, `handoff_holder_mismatch`, `seal_serial_changed`, `custody_gap`, `printed_without_grant`, `opened_before_window`, `key_not_destroyed`.

**Technical term:** Event sourcing / projection.

> **Viva:** "Package state is never stored — it is projected by replaying events, so a missing handover shows up as a visible gap instead of a plausible-looking timeline."

**Remember:** (1) State is calculated, not stored. (2) Seven anomaly types. (3) A gap in the record is itself a finding.

### Feature 6 — Chain integrity verification and Merkle anchors

**What is it?** A button that re-checks the whole chain, plus a way to prove one single record to an outsider.

**What does it do?** It recalculates every link and reports any break.

**Why do we need it?** To prove the record is intact without asking anyone to trust us.

**How is it used?**

```
Operator opens Integrity page → clicks verify → system recomputes all hashes → reports intact or lists breaks
```

**Real-life example:** Counting cash in a till against the receipt roll at closing time.

**What happens technically?** `/verify/chain` recomputes `SHA256(prevHash ‖ bodyHash)` for every entry. A **Merkle tree** (RFC 6962) also lets you prove one event belongs to a day's records by showing only about 20 hashes instead of all events — so you prove one fact without revealing the others.

**Technical term:** Merkle tree; inclusion proof.

> **Viva:** "We recompute the hash chain on demand, and daily Merkle roots let a third party verify that one specific event was included without seeing any other event."

**Remember:** (1) Anyone can recheck. (2) Merkle proof reveals only one record. (3) Proof size grows very slowly (log n).

## 3 · How the whole project works (the story)

A user opens the website. They land on the animated landing page, read what Mohar does, and click **"Open Control Room"**. The dashboard loads and immediately asks the backend for live numbers. The backend reads the database, calculates the answers, and sends them back. The dashboard draws them on screen. Nothing the user does on this screen ever changes the stored history.

```
USER
 ↓  opens http://localhost:5173
BROWSER loads the React app (the Frontend)
 ↓  user clicks "Open Control Room" → /overview
FRONTEND asks for data
 ↓  HTTP request:  GET /api/summary
VITE DEV SERVER forwards /api → http://localhost:8081
 ↓
BACKEND (Fastify, services/ledger) receives it
 ↓  runs SQL
DATABASE (PostgreSQL) returns rows from led.event, led.access_attempt, ref.*
 ↓
BACKEND turns rows into counts and JSON
 ↓  HTTP response (JSON)
FRONTEND receives JSON and updates the screen
 ↓
USER sees packages, refusals, keys and chain tip
```

### Every arrow explained simply

| Arrow | What is really happening |
|---|---|
| User → Browser | The person types the address; the browser downloads the web page files. |
| Browser → Frontend | React builds the visible screen out of small reusable pieces called components. |
| Frontend → API request | JavaScript calls `fetch()` asking the backend a question, e.g. "give me the summary". |
| Vite proxy | In development the frontend runs on port 5173 and the backend on 8081. Vite forwards anything starting with `/api` so the browser does not hit a cross-origin problem. |
| Backend → Database | Fastify runs an SQL query using the `pg` library. |
| Database → Backend | PostgreSQL returns matching rows. |
| Backend → Frontend | The backend converts rows into JSON, a simple text format both sides understand. |
| Frontend → User | React re-renders and the numbers appear. It also re-asks every 10 seconds so the screen stays live. |

### The writing flow (how a record gets in)

```
Field device (has a private key)
 ↓  builds the event, converts to canonical JSON, signs it with Ed25519
POST /events   (or /events/batch when it was offline)
 ↓
LEDGER checks the signature, computes bodyHash, reads the last chain hash
 ↓
INSERT into led.event with hash = SHA256(prevHash ‖ bodyHash)
 ↓
Record is now permanent — the app's database user has no UPDATE or DELETE right
```

## 4 · Frontend, Backend, Database

### Frontend — "the part the user can see and interact with"

In this project the frontend is the **control room**, a React application in `apps/control-room`. It has seven screens plus a landing page. It only **reads and displays**; it never creates a signed event, because writing to the chain requires a device's private key and a browser is not a trusted device.

### Backend — "the part working behind the screen"

The backend is the **ledger service** in `services/ledger`, built with Fastify. It receives requests, checks rules, talks to the database, runs the access engine, calculates the custody projection, and returns JSON.

### Database — "the place where information is stored"

PostgreSQL. It has three groups of tables:

| Group | Tables | What it holds |
|---|---|---|
| `led` (ledger) | `event`, `access_attempt`, `access_key`, `anchor`, `custody_stage` | The permanent history: every act, every access attempt, key records, daily Merkle roots, the 8 stage definitions |
| `ref` (reference) | `authority`, `exam`, `centre`, `package`, `person`, `device`, `roster` | The facts that can change: who exists, which centres, which exams, which packages |
| `fp` (fingerprint) | `copy`, `attribution` | Printed-copy fingerprinting tables |

### Restaurant analogy (adapted to this project)

| Restaurant | Mohar |
|---|---|
| Customer | Control-room operator |
| Menu | Control room screens (React) |
| Waiter | The API (HTTP requests to port 8081) |
| Kitchen | Ledger service (Fastify) |
| Storage | PostgreSQL database |

> **Where the analogy breaks — say this and you will sound sharp:** in a normal restaurant the kitchen can throw away or change stock. Here the kitchen is only allowed to **add** to storage. The database user the backend logs in as has `INSERT` and `SELECT` permission on the event table and **no UPDATE and no DELETE**.

## 5 · API — from zero

**An API is a way for different parts of software to communicate.**

**Real-life example:** You do not walk into the kitchen to cook. You tell the waiter what you want, in a fixed format ("one plate of rice"). The waiter takes it to the kitchen and brings back a result. The waiter is the API — a fixed, agreed way of asking.

```
FRONTEND  →  API request   →  BACKEND  →  processing  →  DATABASE
                                                            ↓
FRONTEND  ←  JSON response ←  BACKEND  ←  rows       ←──────┘
```

| Step | Simple meaning |
|---|---|
| API request | The frontend asks a question at a fixed address, using GET (read) or POST (send/do). |
| Backend | Checks what was asked and whether it is allowed. |
| Processing | Runs the rules — for example the 15 access checks, or replaying events. |
| Database | Stores or fetches the rows. |
| Response | The answer comes back as JSON — text that looks like `{"ok":true}`. |

### The actual APIs in this project

Base address in development: `http://localhost:8081`. All of these were read directly from the route files.

| Method | Endpoint | Input | What it does | Output |
|---|---|---|---|---|
| GET | `/health` | — | Checks the service and returns the newest chain entry | `{ok, chainTip}` |
| GET | `/summary` | optional examId | Counts packages, attempts, keys, devices | counts JSON |
| GET | `/activity` | filters | Merges events + access attempts into one evidence list | activity array |
| GET | `/packages` | optional examId | Lists packages with risk score | package array |
| GET | `/packages/:id` | package id | One package: projection + full timeline | package detail |
| POST | `/packages/:id/declared-state` | state | Records the operator's *planned* state (not the observed one) | status |
| GET | `/access/epoch` | — | Current 6-hour epoch and time remaining | epoch JSON |
| GET | `/access/stages` | — | The 8 custody stages | stage list |
| POST | `/access/request` | key, device, person, geo, seal | **Runs the 15 checks** and records the attempt | granted/denied + evidence |
| GET | `/keys` | filters | Lists custody keys (never the secret) | key list |
| POST | `/keys/issue` | packageId, stage | Creates a key for this epoch | key (shown once) |
| POST | `/keys/rotate` | — | Issues keys for the current epoch | counts |
| POST | `/keys/:id/revoke` | reason | Cancels a key early | status |
| POST | `/events` | signed event | Appends one record to the chain | seq + hash |
| POST | `/events/batch` | signed events | Uploads a queue collected offline | results |
| GET | `/devices` | — | Lists enrolled signing devices | device list |
| POST | `/devices` | pubkey, kind | Enrols a device | device |
| POST | `/devices/:id/revoke` | — | Blocks a device from future signing | status |
| GET | `/exams` `/centres` `/persons` | — | Reference data used by the dashboard | lists |
| GET | `/verify/chain` | fromSeq, toSeq | **Recomputes every hash** and reports breaks | intact + breaks |
| GET | `/verify/inclusion/:eventId` | event id | Merkle proof that one event is in the tree | proof |
| GET | `/anchors` | — | Lists daily Merkle roots | anchor list |
| POST | `/anchors/build` | optional day | Builds that day's Merkle tree | day + treeSize |

## 6 · Technology stack

| Technology | What is it? | Why is it used here? |
|---|---|---|
| TypeScript | JavaScript with type checking | Catches mistakes before the code runs — important when handling evidence |
| React 18 | Library for building user interfaces from components | Builds the control-room screens |
| Vite 5 | Development server and build tool | Fast reloading while developing; also proxies `/api` to the backend |
| React Router 6 | Handles page addresses inside one web app | Gives each screen its own URL, e.g. `/packages` |
| Leaflet 1.9 | Map library | Shows exam centres on a map |
| GSAP + ScrollTrigger | Animation library | Animates the landing page only |
| Node.js | Runs JavaScript on the server | Runs the backend |
| Fastify 5 | Web framework for building APIs | Receives requests and sends JSON; chosen for speed and built-in validation |
| PostgreSQL 18 | Relational database | Stores everything; its permission system is what enforces append-only |
| `pg` | PostgreSQL driver for Node | Lets the backend run SQL |
| zod 3 | Data validation library | Rejects malformed input before it reaches the database |
| pino | Logging library | Records what the server did |
| @noble/hashes | Hashing library | SHA-256 for the chain and key hashing |
| @noble/curves | Cryptography library | Ed25519 signing and verification |
| @noble/ciphers | Encryption library | Present in crypto-core for bundle encryption |
| canonicalize | RFC 8785 JSON canonicaliser | Makes JSON byte-stable so signatures verify |
| shamir-secret-sharing | Splits a secret into parts | 3-of-4 key splitting (**code present, not connected to the unlock flow**) |
| pnpm + Turbo | Package manager and build orchestrator | Manages the many folders (monorepo) and builds them in the right order |

> **Why these choices?** Everything is free and open source. The project records this as a rule in `docs/adr/0004-no-paid-dependencies.md` — no paid API, no cloud service, no hardware security module — because a per-centre licence cost would make district-scale deployment impossible.

## 7 · Feature → technology mapping

| Feature | Frontend | Backend | Database / Other |
|---|---|---|---|
| Hash-chained ledger | Integrity page (React) | Fastify + `@noble/hashes` | `led.event`; grants enforce append-only |
| Digital signatures | — | `@noble/curves` + `canonicalize` | `ref.device` holds public keys |
| Custody keys | Keys page + epoch bar | `domain/keys.ts` | `led.access_key` (hash only) |
| Access engine | Activity page | `domain/policy.ts` | `led.access_attempt` |
| Custody projection | Packages + Workflow pages | `domain/custody.ts` | Replays `led.event` |
| Chain verification | Integrity page | `/verify/chain` | `led.event`, `led.anchor` |
| Centre map | Leaflet | `/centres` | `ref.centre` |
| Landing page | React + GSAP | reads `/summary` | — |

## 8 · Animations and UI

**What is animation?** Making something on screen change smoothly over time instead of jumping instantly.

**What is GSAP?** GreenSock Animation Platform — a JavaScript library that animates elements smoothly and reliably across browsers.

**What is ScrollTrigger?** A GSAP add-on that starts an animation when the user scrolls to a certain point.

**Why are they used?** Only on the **landing page**, to make the product introduction feel polished. The control-room dashboard is deliberately **not** animated, because an operator reading it for hours needs stillness, not motion.

```
User scrolls
 ↓
ScrollTrigger detects the section entering the screen
 ↓
GSAP starts the animation
 ↓
Element fades in and moves up into place
```

| Concept | Simple meaning | Where it is used here |
|---|---|---|
| Scroll animation | Animation triggered by scrolling | Every section reveals as you reach it |
| Stagger | Items animate one after another, not together | Feature cards appear one by one |
| Easing | Speed curve — starts fast, slows down | `power3.out` on reveals |
| Parallax | Background moves less than foreground, creating depth | Hero follows the mouse slightly |
| Pinned horizontal scroll | Section sticks while content slides sideways | Feature showcase section |
| Magnetic button | Button leans toward the cursor | All CTA and Sign In / Sign Up buttons |

> **Accessibility note worth mentioning:** the landing page respects `prefers-reduced-motion`. If a user has asked their computer to reduce motion, everything still appears but nothing moves.

## 9 · Security

Only mechanisms that actually exist in the code are listed. Missing ones are stated honestly.

### 9.1 Append-only enforced by database permissions

**What is it?** The backend's database account is not allowed to change or delete records.
**Why needed?** If the program could edit history, then anyone who breaks into the program could edit history.
**How it works?** `grant select, insert on led.event to mohar_app;` — there is no UPDATE or DELETE grant. A separate account, `mohar_migrator`, is used only to create tables.
**Example:** A postbox — you can drop letters in, you cannot take them out.

> **Viva:** "Append-only is enforced by PostgreSQL role grants, not by application code, so even full compromise of the service cannot rewrite history."

### 9.2 Hash chain (tamper evidence)

**What is it?** Each record carries the fingerprint of the previous one.
**Why needed?** To detect changes made directly in the database by a powerful attacker.
**How it works?** `hash = SHA256(prevHash ‖ bodyHash)`. Editing record 50 breaks 51, 52, 53 and everything after.
**Example:** Numbered pages glued together — remove one and the glue shows.

> **Viva:** "We do not claim records cannot be altered; we claim alteration cannot be hidden."

### 9.3 Ed25519 digital signatures

**What is it?** Proof of which device created a record.
**Why needed?** Hashes prove integrity, not authorship.
**How it works?** The device signs the canonical JSON body with a private key that never leaves it; the server verifies with the stored public key.
**Example:** A signature on a cheque.

> **Viva:** "Every event is attributable to an enrolled device through an Ed25519 signature over RFC 8785 canonical JSON."

### 9.4 Custody keys — hashed, scoped, time-limited

**What is it?** Short codes that expire after six hours.
**Why needed?** To limit the damage of a stolen or shared key.
**How it works?** Only SHA-256 of the key is stored, comparison is **constant-time** (so an attacker cannot guess it letter by letter by measuring response time), and validity is checked by comparing epoch numbers.
**Example:** An OTP that stops working after a few minutes.

> **Viva:** "Keys are stored only as SHA-256 hashes and verified in constant time, and they expire by arithmetic so the system fails closed."

### 9.5 Deny-by-default access engine

**What is it?** Fifteen checks, all must pass.
**Why needed?** Opening a paper bundle is the highest-risk action in the system.
**How it works?** Every check runs; the attempt is written to `led.access_attempt` before the response is returned, and that table also has only SELECT and INSERT grants, so denials cannot be deleted.
**Example:** A bank locker needing two keys, ID, and a signature — all of them.

> **Viva:** "Access is deny-by-default with fifteen non-short-circuiting checks, and refusals are recorded before the caller learns the outcome."

### 9.6 Evidence instead of verdicts

**What is it?** The system stores what it measured, not what it concluded.
**Why needed?** A conclusion cannot be re-examined later; a measurement can.
**How it works?** It stores `"412 m from centre boundary; geofence 150 m"` instead of `"outside_geofence"`.
**Example:** A doctor writing "temperature 39.4 °C" instead of just "unwell".

> **Viva:** "We record evidence rather than verdicts, so a refusal can be re-examined years later and can be annexed to an FIR."

### 9.7 Merkle anchors

**What is it?** A daily summary fingerprint of all that day's records.
**Why needed?** So an outsider can check one record without seeing all of them.
**How it works?** RFC 6962 Merkle tree; `/verify/inclusion/:eventId` returns the proof.

**NOT implemented — say this honestly if asked**

- **Authentication / login.** There is no sign-in system, no user table, no password handling, no sessions, no JWT. The `gateway` service folder is empty. **The API is currently unauthenticated.**
- **Authorisation / protected routes.** Not implemented, because there is no login.
- **RFC 3161 external timestamping.** Anchors are built and stored, but never sent to an outside timestamping authority. The `notarised` flag is never set to true.
- **Rate limiting.** Not present on any endpoint.
- **Device attestation.** Devices are enrolled but their hardware identity is not verified.
- **Shamir 3-of-4 in the live unlock flow.** The code exists in `crypto-core` but is not connected to the running system.
- **Time-lock encryption (drand/tlock) and the ESP32 hardware.** Designed in the docs, not built.
- **HTTPS/TLS.** Development runs over plain HTTP on localhost.

## 10 · Important terms

| | | | |
|---|---|---|---|
| **Frontend** | What the user sees. | **Backend** | What works behind the screen. |
| **API** | A fixed way for software parts to talk. | **Database** | Where information is stored. |
| **Component** | A reusable piece of UI. | **Route** | The address of a page. |
| **Endpoint** | One specific API address. | **JSON** | Simple text format for sending data. |
| **Hash** | A fixed-length fingerprint of data. | **SHA-256** | The specific hash function used here. |
| **Hash chain** | Records linked by including the previous fingerprint. | **Append-only** | You can add, never edit or delete. |
| **Digital signature** | Proof of who created data. | **Ed25519** | The signature method used here. |
| **Private key** | Secret, used to sign. | **Public key** | Shared, used to verify. |
| **Canonical JSON** | JSON always written in one fixed order. | **Epoch** | A fixed 6-hour time block. |
| **Merkle tree** | Hashes combined in pairs up to one root. | **Inclusion proof** | Proof one record is in the tree. |
| **Projection** | State calculated by replaying events. | **Anomaly** | Something the system flags as wrong. |
| **Geofence** | An allowed circle on the map. | **Clock skew** | Difference between two clocks. |
| **Authentication** | Checking who you are. *(not in this project)* | **Authorisation** | Checking what you may do. *(not in this project)* |
| **Monorepo** | Many projects in one repository. | **Deny by default** | Refuse unless every check passes. |

## 11 · User journey

| Step | What happens |
|---|---|
| 1 | User opens `http://localhost:5173` and the **landing page** loads. |
| 2 | The landing page quietly calls `GET /api/summary` to show real live counts. |
| 3 | User clicks **Open Control Room** → React Router moves to `/overview`. |
| 4 | The dashboard loads and calls `/health`, `/summary`, `/access/epoch`. It repeats these every 10 seconds. |
| 5 | Fastify receives each request and runs SQL through `pg`. |
| 6 | PostgreSQL returns rows from `led.event`, `led.access_attempt`, `ref.*`. |
| 7 | The backend calculates counts and returns JSON. |
| 8 | React updates the screen — packages, refusals, unresolved acts, chain tip. |
| 9 | User clicks **Workflow** and picks a package → `GET /packages/JPR-002`. |
| 10 | Backend replays that package's events, builds the projection, finds anomalies, returns state + timeline. |
| 11 | User reads each event with actor, device, position, clock skew, hashes, and raw payload. |
| 12 | User opens **Integrity** and verifies the chain → `GET /verify/chain` recomputes every hash. |

## 12 · Advantages and limitations

### Advantages

- History cannot be silently edited — enforced by the database, not just by code.
- Every record is attributable to a specific enrolled device.
- A missing record shows as a gap instead of a believable timeline.
- Anyone can independently recompute the chain and verify it.
- Access needs 15 checks, and refusals are permanently recorded.
- Keys expire without any background job, so failure denies rather than allows.
- Entirely free and open-source — no paid service anywhere.

### Limitations (be honest)

- **No login or user accounts.** The API is unauthenticated.
- Only the control room is a working app; field and centre apps are empty folders.
- Merkle anchors are stored but never externally timestamped.
- No rate limiting on any endpoint.
- Devices are enrolled but not hardware-verified.
- Shamir key splitting is written but not connected.
- Hardware (ESP32) is designed on paper only.

- Works offline: devices sign locally and upload later; duplicate uploads are safe.

- It cannot stop a leak by someone with legitimate access at the moment of opening.
- Runs on localhost over plain HTTP; not deployed.

## 13 · Future scope

> **NOT CURRENTLY IMPLEMENTED — POSSIBLE FUTURE IMPROVEMENTS**

- **Authentication and roles** — add login so each officer has an account, in the empty `gateway` service.
- **RFC 3161 timestamping** — send each daily Merkle root to a free public timestamp authority so the anchor is provable outside our own database.
- **Field and centre apps** — build the courier and superintendent screens that currently exist only as folders.
- **Connect Shamir 3-of-4** — wire the existing key-splitting code into the real unlock flow.
- **ESP32 room monitor** — the designed hardware for door, presence and footfall sensing.
- **Rate limiting and device attestation.**
- **Deployment with HTTPS.**

## 14 · Viva questions and answers

### Very easy

**1. What is your project?**

> A system that keeps a permanent, tamper-evident record of who handled exam question papers at every stage.

**2. Why did you make it?**

> Because exam paper leaks are usually detected but almost never convicted — the custody record cannot be proved in court.

**3. What problem does it solve?**

> It turns an unprovable paper trail into digital evidence that is signed, permanent, and independently checkable.

**4. Who uses it?**

> Exam boards — district officers, couriers, custodians, centre superintendents, and control-room operators.

**5. What are the main features?**

> An append-only ledger, digital signatures, six-hour custody keys, a fifteen-check access engine, automatic anomaly detection, and chain verification.

### Basic technical

**6. What is frontend?**

> The part the user sees. Here it is the React control room with seven screens plus a landing page.

**7. What is backend?**

> The part working behind the screen. Here it is a Fastify service that checks rules and talks to the database.

**8. What is an API?**

> A fixed way for software parts to communicate. Our frontend calls addresses like `/summary` and gets JSON back.

**9. What is a database and which one did you use?**

> Where information is stored. We use PostgreSQL, with schemas for the ledger, reference data, and fingerprinting.

**10. Why did you choose these technologies?**

> TypeScript for safety, Fastify for a fast validated API, PostgreSQL because its permission system is what actually enforces append-only, and everything free because per-centre licence costs would block district-scale use.

### Project specific

**11. What does "append-only" mean and how do you enforce it?**

> Records can be added but never changed or deleted. We enforce it with PostgreSQL grants — the application's database user has INSERT and SELECT but no UPDATE or DELETE.

**12. How does the hash chain work?**

> Each record's hash is SHA-256 of the previous record's hash joined with its own body hash, so editing an old record breaks every hash after it.

**13. Why two hashes per event?**

> `bodyHash` proves the content is unchanged and is what gets signed; the chain `hash` proves the record's position in history.

**14. What is a custody key?**

> A short code scoped to one package and one stage, valid for a single six-hour epoch, stored only as a SHA-256 hash.

**15. How do keys expire without a scheduler?**

> The epoch is calculated as `floor(unixSeconds / 21600)` and compared at request time. Nothing needs to run, so nothing can fail and leave keys valid.

**16. What are the 15 checks?**

> Key presented, valid, in window, stage match; device enrolled and bound; person on roster and role permitted; geofence and GPS accuracy; custody window; clock skew; seal serial; package state; exam active.

**17. Why do all checks run even after one fails?**

> Because an attempt failing four checks is a different event from one failing a clock skew, and there is no second chance to observe an attempt that already happened.

**18. What is the custody projection?**

> Package state is not stored — it is recalculated by replaying every event, so it can never disagree with the evidence.

**19. What is a custody gap?**

> More than six hours in transit with no recorded holder. The silence itself is reported as a finding.

**20. Why did you remove severity labels like "critical"?**

> Because the severity is invented while the facts are not. We show the distance, the epochs, the skew, and one boolean — whether a human still needs to act.

## Slightly advanced

**21. Why not use blockchain?**

> Blockchain buys agreement among parties who do not trust each other. An exam board is a legal authority, so that problem does not exist. We use the same cryptography — hash chains, Merkle trees, signatures — without the consensus cost, and we work offline.

**22. What stops your own admin from editing records?**

> Nothing stops a database superuser, but nothing hides it either — the edit breaks every later hash and invalidates the day's Merkle root.

**23. Why canonical JSON?**

> Because the same data written in a different key order produces a different hash, which would make a valid signature fail. RFC 8785 fixes one byte representation.

**24. What is a Merkle tree used for here?**

> To prove one event was included in a day's records without revealing any other event — about twenty hashes instead of the whole ledger.

**25. What happens when the network is down?**

> Devices sign and queue events locally and upload later. Each event carries a unique id, so retrying is safe and never duplicates.

**26. Does your project have login?**

> No. Authentication is not implemented — the gateway service is an empty scaffold and the API is currently unauthenticated. It is our main known gap.

**27. Does it prevent leaks?**

> No, and we do not claim it. Someone with legitimate access can still photograph a paper. What we change is that every act becomes attributable and the exposure window narrows.

## 15 · Presentation script

**1. Introduction —** "Good morning. Our project is called Mohar, which means a seal. It is a custody-tracking system for examination question papers."

**2. Problem —** "Between 2015 and 2026 there were about 148 exam-fraud cases across 21 states, and only one conviction. Roughly 70% were paper leaks. The leaks were detected — but the cases failed, because who held the papers was recorded in handwritten registers and phone photos, which cannot be proved in court."

**3. Solution —** "So we did not build a leak detector. We built an evidence system. Every handover is digitally signed by the person's device and written into a ledger where nothing can be edited."

**4. How it works —** "Each record contains the fingerprint of the previous record. If someone edits an old entry, every later fingerprint stops matching, and the tampering becomes visible to anyone who checks."

**5. Main features —** "An append-only ledger, Ed25519 signatures, custody keys that expire every six hours, an access engine that runs fifteen checks before a package can be opened, automatic anomaly detection, and one-click chain verification."

**6. Technology —** "React and Vite on the frontend, Fastify and Node on the backend, PostgreSQL for storage, and TypeScript throughout. Everything is free and open source."

**7. Architecture —** "The frontend only reads. Writing to the chain needs a device's private key, and a browser is not a trusted device. Append-only is not enforced by our code — it is enforced by PostgreSQL permissions, so even if our service is fully compromised it still cannot rewrite history."

**8. Security —** "Hash chaining for tamper evidence, signatures for attribution, hashed and time-limited keys, deny-by-default access, and we store evidence rather than verdicts — 412 metres, not 'outside geofence' — because a measurement can be re-examined in court and a label cannot."

**9. Demo —** "Let me show you a package with an eighteen-hour custody gap." *(see the demo script below)*

**10. Advantages —** "It is tamper-evident, attributable, independently verifiable, works offline, and costs nothing to run."

**11. Limitations —** "We are honest about what is missing. There is no login yet — the API is unauthenticated. Our Merkle anchors are not externally timestamped. And no custody system can stop someone with legitimate access from photographing a paper. We narrow the window and make it attributable; we do not claim to make exams leak-proof."

**12. Conclusion —** "Mohar turns a paper trail that collapses under cross-examination into cryptographic evidence that does not. Thank you."

## 16 · Demo script

| # | Do this | Say this |
|---|---|---|
| 1 | Open `http://localhost:5173` | "This is the landing page. These counters are live from our database, not screenshots." |
| 2 | Scroll through the sections | "It explains the problem and the seven surfaces of the control room." |
| 3 | Click **Open Control Room** | "This takes us into the actual application at `/overview`." |
| 4 | Point at the dashboard | "What is moving, what was refused, and what nobody has resolved. It refreshes every ten seconds." |
| 5 | Open **Workflow**, pick `JPR-002` | "This is one package's entire life, event by event." |
| 6 | Scroll to the custody gap | "Eighteen hours in transit with nobody recorded as holding it. The system reports the silence — that gap is the finding." |
| 7 | Expand one event's details | "Under each event: who, which device, GPS position, clock skew, the body hash, the chain hash, and the raw payload." |
| 8 | Open **Activity**, find a denied attempt | "A refusal. Notice it says the actual distance and the epoch numbers — evidence, not a severity label." |
| 9 | Open **Keys**, show the epoch bar | "These rotate in about two hours. Nothing rotates them — they expire because time passed. If our servers die, keys still expire." |
| 10 | Open **Integrity**, click verify | "This recomputes every hash in the chain live and reports whether it is intact." |
| 11 | Explain what a break would mean | "If this ever failed, it would mean someone edited the database directly — and that is exactly what we want to be impossible to hide." |

> **Before the demo:** start the backend (`node services\ledger\dist\index.js` with `DATABASE_URL` set) and the frontend (`npx.cmd vite`). Check `http://localhost:8081/health` returns `ok:true`. Keep a browser tab already open as a backup.

## 17 · Final one-minute answer

**"Tell me about your project."**

"My project is called Mohar, which means a seal. It is a custody-tracking system for examination question papers.

The problem is that exam paper leaks in India are usually detected but almost never convicted — around 148 documented cases produced one conviction — because the record of who held the sealed papers is kept on handwritten registers and phone photos, which does not survive cross-examination.

So instead of building a leak detector, we built an evidence system. Every time a package changes hands, that act is digitally signed by the person's device and written into an append-only ledger where each record carries the fingerprint of the record before it. If anyone edits old history, every later fingerprint breaks and the tampering becomes visible. And append-only is not just a promise in our code — it is enforced by PostgreSQL permissions, so our own service has no ability to delete or update.

The main features are that ledger, Ed25519 signatures, custody keys that are scoped to one stage and expire every six hours, an access engine that runs fifteen checks before a package can be opened, automatic anomaly detection like custody gaps and seal mismatches, and one-click verification of the whole chain.

Technically it is React with Vite on the frontend, Fastify on Node for the backend, PostgreSQL for storage, all in TypeScript, and everything free and open source.

And to be honest about scope — it does not make exams leak-proof, and we have not built authentication yet. What it does is make every act attributable and every change visible."

---